

Memory in GenAI / LLM Systems — Study Notes

A first-principles walkthrough of why LLMs have no inherent memory, how short-term memory is built using the context window and in-context learning, its limitations, and why and how long-term memory (episodic, semantic, procedural) is built around LLMs — including a four-step architecture (creation, storage, retrieval, injection) and the emerging tooling landscape.

1 Why Memory Matters

Memory is foundational to GenAI applications — no chatbot or agent can function well without it. This topic is framework-agnostic: the goal is a general understanding of memory as a concept, not tied to any single library.

2 LLMs at Inference: A Parameterized Math Function

An LLM at inference is just a parameterized math function: $y = f_{\theta}(x)$.

A **parameterized function** is one whose output depends not only on its input but also on additional parameters. A simple analogy is linear regression, $y = mx + b$: the parameters m (slope) and b (intercept) are learned from data during training, while x is provided by the user at prediction time.

For an LLM, this same structure applies:

- θ — represents billions of learned parameters, fixed after training. The user has no control over θ at inference time.
- x — the input tokens (i.e., the prompt) sent to the LLM. This *is* controllable by the user, which is why changing the prompt changes the output.
- y — the output tokens, depending on both θ and x .

2.1 LLMs Are Stateless by Design

A system is **stateless** if its output depends only on the current input and not on anything that happened before.

Since θ is fixed and each call only uses the current x , an LLM invoked twice — once with prompt x_1 (e.g., "My name is Nitesh") and once with x_2 (e.g., "What is my name?") — treats each call independently. The second call has no knowledge of the first; the LLM will not remember the name shared previously. This confirms LLMs are stateless: each inference call is unique and unrelated to prior calls.

3 The Core Problem: No Intrinsic Memory

Since LLMs are stateless at inference, they have no intrinsic memory — they cannot recall past conversations on their own. Yet virtually every real-world GenAI application (chatbots, agents) requires memory; without it, the user experience is highly frustrating, since every past context must be repeated. This creates a deadlock: LLMs have no built-in memory, but applications cannot function without it.

The resolution: since memory cannot come from inside the LLM, it must be built *externally* — as a system wrapped around the LLM that acts like memory.

4 Two Foundational Concepts

4.1 Context Window

The **context window** is the amount of text an LLM can read and remember at one time before answering.

A useful analogy: the context window is like a camera lens — a bigger lens captures more of the scene, just as a larger context window lets an LLM process more text before responding. Modern LLMs typically have context windows of 128K tokens or more (some, like certain Gemini models, support up to 1 million tokens — roughly equivalent to a 200-page PDF). The key implication: a large context window means a lot of information can be packed into x .

4.2 In-Context Learning

During training, an LLM absorbs patterns from huge amounts of data, storing that knowledge in its parameters (*parametric knowledge*). At inference, it normally searches this parametric knowledge to answer a prompt.

In-context learning is an emergent ability that allows an LLM to use information and patterns present in the prompt itself, in addition to its trained parametric knowledge, to generate an answer.

Example: Pasting a 100-page private company PDF into a prompt and asking a question about it — the LLM has never seen this data during training, but it can still answer correctly by reading and reasoning over the content supplied in the prompt.

5 Building Short-Term Memory

Combining these two concepts yields a simple solution: whenever the LLM is invoked, send the **entire conversation so far**, concatenated, as the input.

Formally: instead of computing $y_2 = f_\theta(x_2)$ for the second turn, compute $y_2 = f_\theta(\text{concat}(x_1, y_1, x_2))$. Because the context window is large enough to hold this, and because in-context learning lets the LLM read x_1/y_1 to answer x_2 , the LLM can now correctly answer something like "What is my name?" after being told the name earlier — not because it "remembers," but because the full history was resupplied as input.

5.1 Implementation

- A **messages** list is maintained, starting with the first prompt (x_1).
- Each LLM response (y_1) is appended back into **messages**.
- Each new user message (x_2 , etc.) is also appended before the next invocation.
- This **messages** list — often called a **conversation buffer** — effectively acts as the system's state, turning an otherwise stateless system stateful.
- If the program restarts, **messages** resets to empty, and the LLM "forgets" everything — hence this is called **short-term memory**.

5.2 Short-Term Memory in Chatbots: Threads

Chatbots organize interactions into **conversations** (sessions). Short-term memory is **conversation-scoped** — it does not exist outside a single conversation. Switching to a new conversation resets the message buffer; returning to an old one would show nothing unless it was separately preserved.

A conversation is also called a **thread**. Short-term memory is **thread-scoped** — its scope is limited to a single thread and does not exist beyond it.

Scoping memory this way avoids a single, unbounded message list that would grow too large and incoherent across potentially thousands of conversations.

6 Limitations of Short-Term Memory

6.1 1. Fragility

If the **messages** variable lives only in application memory, a code reset or server crash erases it — all conversational context is lost. **Solution:** attach persistence — connect a database and, using a thread ID, save the messages for each thread before switching away, so they can be reloaded later without losing context.

6.2 2. Context Window Overflow

As a conversation grows very long, resending the entire history on every turn can eventually exceed the LLM's context window, causing incoherent replies or hallucination. A widely used combined solution:

- **Trimming** — only the most recent n messages are sent (e.g., the last 50 of 500), since the most relevant context is usually recent.
- **Summarization** — the older, trimmed-out messages are summarized (via another LLM call) into a compact summary, which is sent alongside the recent messages, keeping the total token count within limits while preserving older context in condensed form.

6.3 3. Thread-Scoping (the most critical limitation)

Because short-term memory is locked to a single thread, several problems emerge:

1. **No continuity across conversations** — e.g., a user's stated preference (like preferring Python over Java) in one conversation is forgotten in the next.

2. **Learning never compounds over time** — effort spent guiding the LLM (e.g., teaching it to use window functions instead of subqueries) must be repeated from scratch in every new conversation.
3. **Cross-thread reasoning is impossible** — the LLM cannot answer "what did we discuss yesterday?" or recall solutions from a previous, separate conversation.

This makes true personalization impossible with short-term memory alone: a personal assistant needs to build a profile of the user's preferences, background, and habits across *many* conversations — something thread-scoped memory cannot support by definition.

7 Long-Term Memory

Long-term memory is memory that exists *outside* and *beyond* a single conversation, persisting for a long period of time (days, weeks, or longer) — as opposed to short-term memory, which is destroyed when a conversation ends.

It needs two key properties:

1. **Durability** — able to store special information that survives beyond a single conversation/session (e.g., facts about the user: their role, nationality, profession; or facts relevant to a long-running task, like writing a book over several months).
2. **Selectivity** — highly selective; it does not blindly store entire conversations, but extracts only the stable, useful, and reusable pieces of information from each conversation.

8 Three Types of Long-Term Memory

1. **Episodic memory** — records what happened in the past (e.g., "the user rejected this solution last session," "these deployment credentials were incorrect," "this approach worked / didn't work before"). It lets the agent answer questions like "what did we do last time?" and avoid repeating past mistakes.
2. **Semantic memory** — the most common and important type; stores **facts** about the user or the system (e.g., "the user prefers Python," "the user is a beginner," "the system uses PostgreSQL," "the budget constraint is 10,000"). This is essentially user-level or system-level factual memory.
3. **Procedural memory** — stores **strategies, rules, and learned behaviors** — i.e., how to do things (e.g., "avoid subqueries when solving SQL problems for this user," "if tool X fails, try tool Y," "always explain things step by step for this user," "the preferred workflow for task Z is..."). This is what makes an agent feel like it improves and adapts to a specific user over time.

9 How Long-Term Memory Works: A Four-Step Architecture

9.1 Step 1: Creation (or Update)

During a conversation, the system must decide whether anything just said is worth remembering beyond this conversation. This involves:

- Extracting **memory candidates** from user messages, model responses, and tool outputs (e.g., "I prefer Python over Java" becomes a candidate).
- Filtering out noise — extracting only the core fact.
- Deciding the **scope** — user-level, app-level, or agent-level.
- Deciding whether to create a new memory, update an existing one, or ignore the candidate entirely.

9.2 Step 2: Storage

Once a piece of information is confirmed worth keeping, it is saved to a **durable store** along with identifiers and metadata for easy future retrieval, so it survives restarts and crashes. The choice of storage depends on the memory type — options include relational databases, key-value stores, log/text files, or vector databases (for semantic search).

9.3 Step 3: Retrieval

When a new conversation begins, before replying, the system asks: "given the current situation, what should I remember right now?" This involves examining the current user input, deciding whether memory is needed, searching the memory store, and pulling back only a **small, relevant subset** of stored memories.

Retrieval is **selective, not exhaustive** — unlike short-term memory (which loads the whole conversation buffer), long-term memory retrieval performs a targeted search and returns only relevant pieces.

9.4 Step 4: Injection

Long-term memory is never used by the LLM directly. Retrieved memory is pulled into the **short-term memory** (the conversation buffer) first, becoming part of the prompt/context window, and only then reaches the LLM as additional input tokens — indistinguishable from any other prompt text.

10 Challenges in Building Long-Term Memory Systems

1. **Deciding what to remember** — filtering signal from noise in normal conversation to decide what deserves to become a long-term memory is genuinely difficult.
2. **Deciding what to retrieve, in real time** — figuring out which subset of a potentially large memory store is relevant to the current conversation is equally challenging.
3. **Orchestrating the entire system** — integrating memory creation, storage, and retrieval into an already complex agentic system adds significant engineering complexity (connecting memory stores, managing many moving parts).

11 Emerging Tools for Memory Management

Several libraries and managed platforms have emerged to handle long-term memory so developers can focus on building the application itself:

- **LangMem** — from the LangChain family, integrates well with LangGraph for building AI agents.

- **Mem0** — a popular platform providing a memory layer for GenAI apps.
- **Supermemory** — another platform performing the same role of managing long-term memory for GenAI applications.

This space is growing quickly, reflecting how central memory is to building capable GenAI and agentic applications.

12 Future Direction: Intrinsic Memory in LLMs

Because all of these external memory systems exist only to compensate for LLMs’ lack of built-in memory, research is underway to build model architectures with **intrinsic memory** — for example, Google’s research on new transformer-style architectures aimed at giving models memory natively, potentially reducing the need for such elaborate external memory systems in the future.

13 Quick Revision Summary

1. An LLM at inference is a parameterized function $y = f_{\theta}(x)$; since θ is fixed and only the current x is used, LLMs are stateless — they have no intrinsic memory.
2. Short-term memory is built externally by resending the full conversation history as input on every call, exploiting a large context window and in-context learning.
3. Short-term memory is implemented via a growing **messages**/conversation-buffer variable, and is conversation- (thread-) scoped.
4. Limitations of short-term memory: it is fragile (lost on crash/reset unless persisted), can overflow the context window (solved via trimming + summarization), and — most critically — is thread-scoped, preventing continuity, compounding learning, and cross-thread reasoning.
5. Long-term memory exists outside a single conversation, and must be durable and selective.
6. Three types: episodic (past events), semantic (facts about user/system), and procedural (strategies/rules/learned behaviors).
7. Long-term memory operates via four steps: creation (deciding what’s worth remembering), storage (saving durably with metadata), retrieval (selectively searching for what’s relevant right now), and injection (pulling retrieved memory into short-term memory/context, never feeding the LLM directly).
8. Key challenges: deciding what to create, deciding what to retrieve, and orchestrating the overall system — driving demand for managed memory-layer tools like LangMem, Mem0, and Supermemory.
9. Long-term, research aims to give LLMs intrinsic memory, reducing reliance on external memory systems altogether.